P3787R0
Adjoints to "Enabling list-initialization for algorithms": uninitialized_fill

Published Proposal, 2025-07-10

Author:
Giuseppe D'Angelo

Audience:
LWG, LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

We extend the same changes that [P2248R8] applied to the rest of the find algorithms to `std::ranges::uninitialized_fill`.

Table of Contents

1. Changelog

- R0
  - First submission

2. Motivation and scope

In the Tokyo 2024 meeting [P2248R8] (Enabling list-initialization for algorithms) was adopted.

Due to an oversight, the `std::uninitialized_fill` family of algorithms was excluded from the ones where a defaulted template parameter was added.

We propose to modify `uninitialized_fill`'s specification, so that it matches the post-P2248 one for the rest of the algorithms (especially `fill`).

3. Proposed Wording

All the proposed changes are relative to [N5008].

In [version.syn], bump the feature-test macro as shown:

```
#define __cpp_lib_algorithm_default_value_type 202403L YYYYMML
  // also in <algorithm>, <ranges>, <string>, <deque>, <list>, <forward_lis
```

where the new value corresponds to the year and month of adoption of the present proposal.

✧ ✧ ✧

Modify [memory.syn] and the corresponding signatures in the entries under [uninitialized.fill] as shown:

```
template<class NoThrowForwardIterator, class T = typename std::iterator_tra
  constexpr void uninitialized_fill(NoThrowForwardIterator first,
                                    NoThrowForwardIterator last, const T& x
template<class ExecutionPolicy, class NoThrowForwardIterator, class T = typ
  void uninitialized_fill(ExecutionPolicy&& exec,
                          NoThrowForwardIterator first,
```

```
                          NoThrowForwardIterator last,
                          const T& x);
template<class NoThrowForwardIterator, class Size, class T = typename std:
  constexpr NoThrowForwardIterator
    uninitialized_fill_n(NoThrowForwardIterator first, Size n, const T& x);
template<class ExecutionPolicy, class NoThrowForwardIterator, class Size,
  NoThrowForwardIterator
    uninitialized_fill_n(ExecutionPolicy&& exec,
                         NoThrowForwardIterator first,
                         Size n, const T& x);

namespace ranges {
  template<nothrow-forward-iterator I, nothrow-sentinel-for<I> S, class T =
    requires constructible_from<iter_value_t<I>, const T&>
      constexpr I uninitialized_fill(I first, S last, const T& x);
  template<nothrow-forward-range R, class T = range_value_t<R>>
    requires constructible_from<range_value_t<R>, const T&>
      constexpr borrowed_iterator_t<R> uninitialized_fill(R&& r, const T& x

  template<nothrow-forward-iterator I, class T = iter_value_t<I>>
    requires constructible_from<iter_value_t<I>, const T&>
      constexpr I uninitialized_fill_n(I first,
                                       iter_difference_t<I> n, const T& x);
}
```

§ 4. Acknowledgements

Thanks to Ruslan Arutyunyan for spotting this oversight.

Thanks to KDAB for supporting this work.

All remaining errors are ours and ours only.

§ References

§ Informative References

[N5008]

Thomas Köppe. *Working Draft, Programming Languages — C++*. 15 March 2025. URL: https://wg21.link/n5008

[P2248R8]

Giuseppe D'Angelo. *Enabling list-initialization for algorithms*. 20 March 2024. URL: https://wg21.link/p2248r8